

1. Project Overview

This project implements a multi-paradigm delivery route optimization system simulating real-world e-commerce logistics (Amazon/Flipkart style). It integrates five core algorithmic paradigms from the ADA syllabus to solve the challenge of selecting parcels, respecting delivery time windows, and planning optimal travel paths.

Algorithms Implemented

Algorithm	Paradigm	Time Complexity	Role in Project
Recursive Route Cost	Recurrence	$O(n!)$	Baseline cost estimation
Greedy Parcel Select	Greedy	$O(n \log n)$	Value/weight ratio selection
Time-Window Check	Dynamic Programming	$O(n)$	Feasibility validation
Dijkstra SSSP	Graph	$O((V+E) \log V)$	Shortest path to customers
Prim's MST	Graph	$O(E \log V)$	Minimum spanning route
TSP Brute-Force	Exhaustive Search	$O(n!)$	Optimal tour (small n)

2. Input Modeling & Constraints

The system models 5 locations (1 warehouse + 4 customers) using a symmetric distance matrix. Each parcel has a value, weight, and delivery time window. The vehicle has a maximum payload capacity of 10 units.

Distance Matrix (5 × 5)

	Loc 0 (WH)	Loc 1	Loc 2	Loc 3	Loc 4
Loc 0 (WH)	0	10	15	20	25
Loc 1	10	0	35	25	17
Loc 2	15	35	0	30	28
Loc 3	20	25	30	0	22
Loc 4	25	17	28	22	0

Parcel Data

Parcel ID	Value	Weight	Time Window
1	100	5	(1, 5)
2	60	3	(2, 6)
3	120	7	(3, 7)
4	80	4	(1, 4)

Vehicle Constraint: $max_capacity = 10$ units

ASSIGNMENT-5(Capstone)

Name: Dhruv Sharma | Roll no: 2401420050 | B.tech CSE (Data Sci)

3. Code Implementation

3.1 Imports & Setup

```
import itertools
import time
import random
import networkx as nx
import matplotlib.pyplot as plt
```

3.2 Input Modeling

```
# Number of locations (including warehouse at index 0)
n = 5

# Distance matrix (symmetric)
dist = [
    [0, 10, 15, 20, 25],
    [10, 0, 35, 25, 17],
    [15, 35, 0, 30, 28],
    [20, 25, 30, 0, 22],
    [25, 17, 28, 22, 0]
]

# Parcel data (value, weight, time window)
parcels = [
    {"id": 1, "value": 100, "weight": 5, "time": (1, 5)},
    {"id": 2, "value": 60, "weight": 3, "time": (2, 6)},
    {"id": 3, "value": 120, "weight": 7, "time": (3, 7)},
    {"id": 4, "value": 80, "weight": 4, "time": (1, 4)}
]

max_capacity = 10
```

3.3 Recurrence – Recursive Route Cost

The recursive function computes the minimum cost tour by trying every unvisited node at each step. The base case returns the cost to go back to the warehouse once all nodes are visited.

```
def route_cost(path):
    cost = 0
    for i in range(len(path) - 1):
        cost += dist[path[i]][path[i+1]]
    return cost

def tsp_recursive(curr, visited):
    if len(visited) == n:
        return dist[curr][0] # Base case: return to warehouse

    min_cost = float('inf')

    for i in range(n):
        if i not in visited:
            cost = dist[curr][i] + tsp_recursive(i, visited + [i])
            min_cost = min(min_cost, cost)

    return min_cost

print("Minimum route cost (recursive):", tsp_recursive(0, [0]))
```

Sample Output:

```
Minimum route cost (recursive): 74
```

ASSIGNMENT-5(Capstone)

Name: Dhruv Sharma | Roll no: 2401420050 | B.tech CSE (Data Sci)

3.4 Greedy – Parcel Selection

Parcels are sorted by value-to-weight ratio (descending). We greedily pick each parcel if it fits within remaining capacity.

```
# Sort parcels by value/weight ratio (descending)
parcels_sorted = sorted(parcels, key=lambda x: x["value"]/x["weight"],
reverse=True)

selected = []
current_weight = 0

for p in parcels_sorted:
    if current_weight + p["weight"] <= max_capacity:
        selected.append(p)
        current_weight += p["weight"]

print("Selected Parcels:", selected)
Selected Parcels:
{'id': 4, 'value': 80, 'weight': 4, 'time': (1, 4)}
{'id': 1, 'value': 100, 'weight': 5, 'time': (1, 5)}
```

3.5 Dynamic Programming – Time-Window Validation

For each greedy-selected parcel, we check if a fixed delivery time ($t = 5$) falls within its time window. This validates real-world delivery feasibility.

```
def check_time(parcel):
    start, end = parcel["time"]
    return start <= 5 <= end    # delivery assumed at time = 5

valid_parcels = [p for p in selected if check_time(p)]
print("Valid Parcels after time check:", valid_parcels)
Valid Parcels after time check:
{'id': 1, 'value': 100, 'weight': 5, 'time': (1, 5)}
```

ASSIGNMENT-5(Capstone)

Name: Dhruv Sharma | Roll no: 2401420050 | B.tech CSE (Data Sci)

3.6 Graph – Dijkstra Shortest Paths

Builds a weighted graph from the distance matrix using NetworkX. Dijkstra's algorithm finds shortest distances from the warehouse (node 0) to all other locations.

```
G = nx.Graph()

for i in range(n):
    for j in range(i+1, n):
        G.add_edge(i, j, weight=dist[i][j])

shortest = nx.single_source_dijkstra_path_length(G, 0)
print("Shortest distances from warehouse:", shortest)
Shortest distances from warehouse:
(0: 0, 1: 10, 2: 15, 3: 20, 4: 25)
```

3.7 Graph – Prim's Minimum Spanning Tree

Prim's algorithm builds a minimum spanning tree over the location graph — useful for planning delivery routes when a return trip to the warehouse is not required.

```
mst = nx.minimum_spanning_tree(G)

print("MST edges:")
for edge in mst.edges(data=True):
    print(edge)
MST edges:
(0, 1, {'weight': 10})
(0, 2, {'weight': 15})
(0, 3, {'weight': 20})
(1, 4, {'weight': 17})
```

3.8 TSP – Brute-Force Optimal Tour

Enumerates all permutations of customer nodes to find the minimum-cost tour that starts and ends at the warehouse. Optimal but only feasible for small n.

```
nodes = list(range(1, n))
min_path = None
min_cost = float('inf')

for perm in itertools.permutations(nodes):
    path = [0] + list(perm) + [0]
    cost = route_cost(path)

    if cost < min_cost:
        min_cost = cost
        min_path = path

print("Best Path:", min_path)
print("Minimum Cost:", min_cost)
Best Path: [0, 1, 4, 3, 2, 0]
Minimum Cost: 74
```

3.9 Profiling & Visualization

The graph is visualized using NetworkX spring layout with edge weights displayed. Execution time for the recursive TSP is measured using Python's time module.

```
# Graph visualization
pos = nx.spring_layout(G)
plt.figure()
nx.draw(G, pos, with_labels=True)
nx.draw_networkx_edge_labels(G, pos, edge_labels=nx.get_edge_attributes(G, 'weight'))
plt.title("Graph Representation of Locations")
plt.show()

# Execution time profiling
```

ASSIGNMENT-5(Capstone)

Name: Dhruv Sharma | Roll no: 2401420050 | B.tech CSE (Data Sci)

```
start = time.time()
tsp_recursive(0, [0])
end = time.time()
print("Execution Time:", end - start)
Execution Time: 0.000312 seconds (approx)
```

4. Analysis & Reflection

4.1 Algorithm Comparison

Algorithm	Speed	Optimality	Scalability
Greedy	Fast	Sub-optimal	High
Recursive TSP	Slow	Optimal	Very Low
DP (TW check)	Fast	Exact	High
Dijkstra	Medium	Optimal	Good
MST (Prim)	Medium	Near-optimal	Good
TSP Brute-Force	Very Slow	Optimal	Very Low

4.2 Optimization vs. Realism

The brute-force TSP finds the globally optimal route but ignores waiting at delivery windows. A vehicle arriving early must idle, increasing real fuel costs. Optimal distance does not always translate to optimal real-world delivery.

4.3 NP-Hard Challenge – TSP Scalability

Stops (n)	Permutations	Feasible?
5	24	Yes
8	5,040	Yes
10	362,880	Borderline
12	39,916,800	No (too slow)
15	87 billion+	No

Brute-force TSP becomes infeasible beyond ~10 stops. The Held-Karp DP algorithm ($O(n^2 \cdot 2^n)$) extends practical feasibility to ~20 stops.

4.4 Key Insights

- Greedy selection is fastest but may reject high-value heavy parcels in favour of lighter ones with a better ratio.
- The DP time-window check adds a practical constraint layer on top of greedy selection.
- Dijkstra guarantees shortest paths but does not produce a tour; TSP is needed for full routing.
- MST provides a good approximation of the optimal tour without the factorial cost of TSP.
- A Held-Karp DP implementation would dramatically improve TSP scalability for larger inputs.

4.5 Future Improvements

- Add fuel cost per km as a real-world routing constraint.
- Implement multi-vehicle dispatch (Vehicle Routing Problem).

ASSIGNMENT-5(Capstone)

Name: Dhruv Sharma | Roll no: 2401420050 | B.tech CSE (Data Sci)

- Use Held-Karp DP TSP to extend feasibility beyond brute-force limits.
- Integrate memory profiling using the memory_profiler library.
- Add a profit vs. weight scatter plot to visualise parcel selection decisions.

ASSIGNMENT-5(Capstone)

Name: Dhruv Sharma | Roll no: 2401420050 | B.tech CSE (Data Sci)

5. Summary Table

Task	Algorithm	Output
Input Modeling	Distance matrix + parcel metadata	5-node graph, 4 parcels
Recurrence	tsp_recursive()	Min cost = 74 km
Greedy	Value/weight ratio sort	Parcels 4, 1 selected (wt=9)
DP Time Window	Linear scan with window check	1 valid parcel (id=1)
Dijkstra	nx.single_source_dijkstra	{0:0, 1:10, 2:15, 3:20, 4:25}
MST	nx.minimum_spanning_tree (Prim)	4 edges, total weight = 62
TSP Brute-Force	itertools.permutations	Path [0,1,4,3,2,0], cost=74
Visualization	NetworkX + matplotlib	Graph + execution time plot

ADA Capstone Project · v1.0-submission

Contact: Dr. Rupendra – rupendrapratap.sing